Lab 4 Report
Name: Muvil Kothari, Weng Hoi, Khoo Yong Xuan

Exercise 1: Search (findLeafPage)

What we implemented: We created a recursive function that starts at the root and travels down the tree until it hits a leaf page.

- **Navigation:** In each internal node, we looped through the entries to find where our search key fits.
- **Null Case:** If the search key was null, we forced the code to always take the leftmost child pointer to support full table scans.
- **Base Case:** Once the page category identified it as a Leaf, we fetched it with the requested permissions and returned it.

Implementation:

- **Handling Duplicates:** We chose to use LESS_THAN_OR_EQ during comparisons. This was a specific choice to ensure that if the same key exists on multiple pages, we always land on the **leftmost** possible leaf containing that key.

Exercise 2: Splitting

What we implemented: We implemented the logic to handle full pages by splitting them into two and keeping the tree balanced.

- **Leaf Split:** We created a new leaf page, moved the second half of the tuples over, and updated the left and right pointers so the leaves stayed linked like a chain. We then copied the first key of the new page up to the parent.
- **Internal Split:** Similar to the leaf, but we pushed up the middle key to the parent instead of copying it. We also had to update the parent pointers of all children that moved to the new page.

Implementation:

- **Snapshot strategy:** When moving tuples or entries, we first copied them into a temporary ArrayList. This was a decision made to prevent "Concurrent Modification" errors basically, we didn't want to be deleting items from the page while the loop was still trying to read them.
- **Double parent link:** We explicitly called setParentId on **both** the original page and the new right hand page after every split. This ensured that even if a split happened deep in the tree, every node always knew who its parent was.

Exercise 3: Redistribution

What we implemented: This was the shrink logic. When a page became less than half full, we checked its siblings to see if they had extra data to share.

- **Leaf steal:** We moved tuples from a rich sibling to the under full page.
- **Internal steal:** We performed a rotation. We pulled the parent's divider key down into the empty node and pushed a key from the sibling up to the parent.

Implementation:

- **Immediate parent updates:** During internal redistribution, we decided to update the parent's divider key **inside the loop** (after every single entry was moved). This was the critical change that passed the system test, it kept the tree legal at every micro-step so SimpleDB's internal safety checks didn't trigger an error.

Exercise 4: Merging

What we implemented: If a page was under full and its neighbors were also poor, we merged them into one.

- **Leaf merge:** We moved all data to the left page, updated the sibling pointers to skip the now empty right page, and deleted the divider from the parent.
- **Internal merge:** We pulled down the parent key to act as a bridge, combined all entries into the left page, and updated all children to point to their new merged parent.

Implementation:

- **Junction pointer check:** In mergeInternalPages, we added a specific check: if (leftRightmost.equals(rightLeftmost)). This was a decision to prevent a duplicate pointer bug where the divider might accidentally separate two identical children, which would have corrupted the tree.

Key Changes

- **Explicit dirtying:** In every method (split, steal, merge) we manually added every affected page (parent, left child, right child) to the dirtypages map. This ensured that the BufferPool never evicted a page while we were in the middle of changing its pointers.
- **Internal page helper:** We had to add a new helper method setChildId to BTreeInternalPage.java. This was necessary because the provided code didn't have an easy way to update the very first child pointer (index 0) during rotations and merges.
- **Try-catch:** We wrapped the main insertion/deletion logic in a try catch block. This ensured that if a transaction failed (like a deadlock), we still marked all touched pages as dirty so the system could recover correctly.